

chap 3: Solving Problems by Searching

(1) Problem-Solving agents:

- They are also known as goal-based agents in AI that decides, what action to take to reach a desired goal by solving a problem systematically.
- Its formulate a problem, searches for a solⁿ and then execute action to reach the goal.
- Different steps involve in solving problem are as follows:

(a) Goal formulation: Define the desired Outcome or goal.

(b) Problem formulation: Define the problem in a Searchable form.

(c) Search: Explore possible actions and path.

(d) Find Solution: Select the best path to the goal.

(e) Execute: Perform the actions from the selected solution.

Example: Romania

- On holiday in Romania; Currently in Arad.
- Flight leaves tomorrow from Bucharest.
 - formulate goal:
 - be in Bucharest
 - formulate Problem:
 - States: Various cities
 - actions: drive between cities
 - Find Solutions:
 - Sequence of cities, Eg. Arad, Sibiu, Iagoras, Bucharest.

(i) Well-defined problems and Solutions:

- These are following five components involved in problem formulated:

• Initial State: It is the starting state or initial step of the agent towards its goal.

• Actions: It is the description of the possible actions available to the agent.

• Transition Model: It describes what each action does.

• Goal Test: It determines if the given state is a goal state.

• Path Cost: It assigns a numeric cost to each path towards its goal. Problem solving agent selects a cost function which reflects its performance measure. Always an optimal solution has to be found out because its based upon lowest path cost, minimum time, reaching to a goal.

(ii) Formulating Problem:

- The formulation problem is an abstract mathematics description is not real. In formulating problem unnecessary details are hidden from the main description.
- Example, Suppose we want to move from location Arad (Initial State) to the goal state Bucharest.
- While moving from source to destination many things come across like travelling companions, the current radio programs, the scenery out of the window, next stop, the condition of the road, the weather

Condition and so on.

- All these are irrelevant things that are not be including by formatting problems.
- The processes of removing these unnecessary details is known as abstraction. So, in problem formulation we just includes different cities names that we come across from initial state to goal state.

(2) Example Problems:

Basically, there are two types of problem approaches:

- Toy Problem: It is a concise and exact description of the problem which is used by the researchers to compare the performance of algorithms.

Example,

- (1) Vacuum - cleaner agent
- (2) 8 - puzzle problem
- (3) 8 - queens problem

- Real World Problem: It refers to a real problem that doesn't have exact solutions, available as like Toy problem.

- Just actions has to be taken in a search of a goal and try to findout a solution.

Example,

- (1) Healthcare
- (2) Public Security
- (3) Education
- (4) Sentiment analysis

(1) Toy Problem:

(a) Eight Vacuum - cleaner Problem:

- This Vacuum - cleaner agent problem can be formulated like:

- (i) States: The state is determined by both the agent location and the dirt location. The agent is in one of two locations, which might or might not contain dirt. Therefore, there are $2 \times 2^2 = 8$ possible world states. A larger environment with 'n' location has $n \cdot 2^n$ states.

- (ii) Initial State: Any state can be designed as the initial state.

- (iii) Actions: In the larger environment, there are six environment, that is, Left, Right, Suck, Noop, Up and down.

- (iv) Transition Model: After taking action the next state that agent will reach is known as transition state. Suppose moving from left to right then right become transition state.

- (v) Goal State: Everytime after taking an action and reaching to next state, that companions, the state is transition state check whether agent has reach to the goal state or not.

- (vi) Path Cost: Each step costs 1, so the path cost is the number of steps in the path.

Refers to
pgs

(b) 8-Puzzle Problem:

Q. Given:

3	7	6
5	1	2
4	-	8

5	3	6
7	-	2
4	1	8

Initial State

Goal State

Formulate using 8-puzzle problem and path cost Problem?

Soln: Step 1: Initial State: $\{(3, 7, 6) (5, 1, 2) (4, 0, 8)\}$.

(i) Up:

3	7	6
5	-	2
4	1	8

$\{(3, 7, 6) (5, 0, 2) (4, 1, 8)\}$

(ii) Up:

3	-	6
5	7	2
4	1	8

$\{(3, 0, 6) (5, 7, 2) (4, 1, 8)\}$

(iii) Left:

-	3	6
5	7	2
4	1	8

$\{(0, 3, 6) (5, 7, 2) (4, 1, 8)\}$

(iv) Down:

5	3	6
-	7	2
4	1	8

$\{(5, 3, 6) (0, 7, 2) (4, 1, 8)\}$

(v) Right:

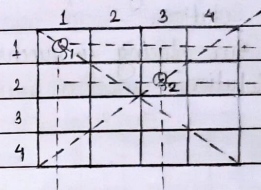
5	3	6
7	-	2
4	1	8

$\therefore$ Reach the Goal State:
 $\{(5, 3, 6) (7, 0, 2) (4, 1, 8)\}$

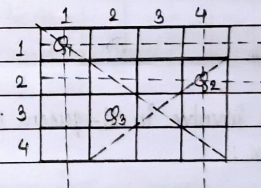
$\therefore$ Therefore, the path cost from Initial State to goal state is equal to 5, that means moving 5 times blank spaces we reach to goal State.

(c) 8-Queens Problem:

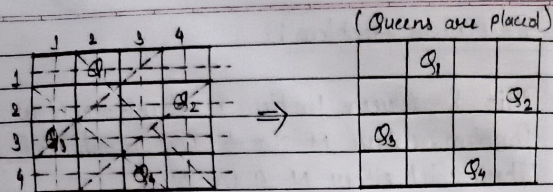
- This 8-Queens Problem is also known as N-Queens Problem, where N stands for number of queens.
- There will either $N=4$ or $N=8$.
- In this problem, if $N=4$ (4×4 matrix), so in this 4×4 matrix queens has to be placed in such a way that number of 2 queens either be arranged (not diagonally or column wise).



- In this situation after placing Q_1 and Q_2 , we are unable to place Q_3 , so we will backtrack and change the position of Q_4 . In order to place further queens on the chess-board.



- In this situation after placing Q_3 , we are unable to place Q_4 . So again, we will backtrack and change the position of Q_3 but can't place it. So again backtrack and change the position of Q_2 but unable to change, so again backtrack to change the position of Q_1 .
- Hence, all the queens are placed.



- In the same way we can place 8-queens on the chess-board.
- Number of solution for placing 8-queens on the chess-board = 92 distinct solution.
- Number of solution for placing 4-queens on the chess-board = 8 distinct solution.

	1	2	3	4	5	6	7	8
1								
2								
3								
4								
5								
6								
7								
8								

- Different components involve in 8-queens problem or N-queens problem are:

- States: Arrangement of any 0 to 8-queens on the chess-board.
- Initial State: An Empty chess-board.
- Actions: Add a queen to any empty box.
- Transition Model: Returns the chess-board with the queen added in a box.

- Goal Test: Checks whether 8-queens are placed on the chess-board without any attack.
- Path Cost: There is no need for path cost because only final states are counted.

(2) Real-World Problem:

- This problem is based on route-finding problem which is defined in terms of specified location with given initial state and goal state.
- Agents have to perform searching from initial state and by taking actions in order to reach specified goal within less time, less cost, so that optimal solution is an obtained.
- Example, consider airline travel problem that is solved by travel planning website.
- It refers to various components like:-
 - (1) States
 - (2) Initial state
 - (3) Actions
 - (4) Transition Model
 - (5) Goal Test
 - (6) path cost

(3) Searching for Solutions:

- After formulating goal, search process starts, action is taken to reach given goal.
- It is based upon different types of search algorithms like: DFS, BFS, A*, AO*, etc.
- It includes:-

Date _____
Page _____

(a) Infrastructure for Search Algorithms:

- It includes:-

(i) n. STATE - The state in the state space to which the node correspondings.

(ii) n. PARENT

(iii) n. ACTION

(iv) n. PATH-COST

(b) Measuring Problem-Solving Performance:

- Any algorithm performance is calculated or checked depending upon four components:-

(i) Completeness:- It refers whether, we have reached to a goal or not.

(ii) Time complexity:- How much time an agent requires to reach a goal.

(iii) Space Complexity:- It refers to maximum no. of nodes in memory.

(iv) Optimality:- If an agent reach to goal, within less time, less cost and even searching space (memory) is not utilized more than we can an algorithm is an Optimal (best).

- Time and Space complexity are measured in terms of:-

(i) b: Maximum branching factor of the search tree.

(ii) d: Depth of the least-cost solutions.

(iii) m: Maximum depth of the state space (may be ∞ (Infinite)).

Date _____
Page _____

(4) Uninformed Search Strategies:

• It is also known as blind search.

• This search have no additional information about the states that provided in the problem definition.

• In this search agent start taking an action from the initial state and then exploring the states in between to reach goal state.

• Different types of Uninformed Search Strategies are as follows:-

a) Breadth-First Search (BFS)

b) Depth-First Search (DFS)

c) Uniform-cost Search (UCS)

d) Depth-limited Search (DLS)

e) Iterative deepening depth-First Search (IDFS)

f) Bidirectional Search.

g) Comparing Uninformed Search Strategies.

(a) Breadth-First Search (BFS):

- BFS is a search algorithm use to find shortest path or solution in a graph or tree like structure.

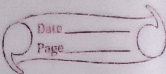
- It explores, level by level, one tree at a time until solution is found.

- It can be implemented in the FIFO queue, data structure.

• Properties of BFS:

(i) Completeness: Yes (if b is finite) branching factor is finite.

b - branch
d - depth

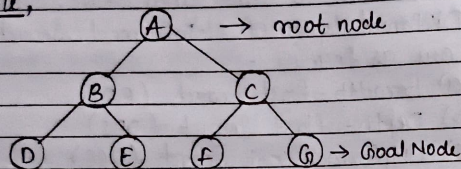


(ii) Time complexity : $O(bd+1)$.

(iii) Space complexity : $O(bd+1)$ (keeps every node in memory).

(iv) Optimal : Yes (if cost = 1 per step).

• Example,



∴ The tree can be traversed as,
 $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$ reach goal node.

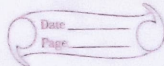
• Advantages of BFS :

- It finds the shortest path (if all steps have equal cost).
- Complete $\rightarrow$ Will find a solution if it exists.
- Simple to implement using a queue.

• Disadvantage of BFS :

- Use lot of memory.
- Time complexity is high.
- Slow for deep or large trees.

(b) Depth - first Search (DFS) :



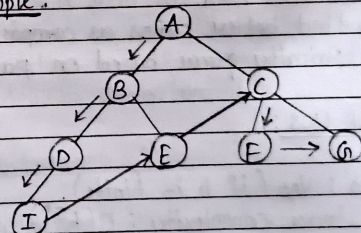
- It is a search algorithm that explores the tree as far as possible along one branch before backtracking.
- It is implemented using LIFO technique.
- It goes into

- Hence, search is incomplete which never back.
- It follows stack data structure.

• Properties of DFS :

- Completeness : DFS doesn't guarantee to reach the goal state.
- Optimality : It doesn't give an optimal solution as it expands nodes in one direction deeply.
- Time complexity : DFS has $O(b^m)$.
- Space complexity : $O(b^m)$ where b is branching factor (that is, total no. of child nodes, a parent node have) and m is the maximum length of any path.

• Example :



∴ In DFS tree can be visited as,
 $A \rightarrow B \rightarrow D \rightarrow I \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

• Advantages of DFS :

- It Uses less memory than BFS.
- It is faster when solution is Deep in the tree.
- Easy to implement using recursion.

• Disadvantages of DFS:

- No guarantee to reach a goal node. (If it reaches infinite loop).
- Due to backtracking technique it becomes slow in some cases.
- No Optimal Solutions.

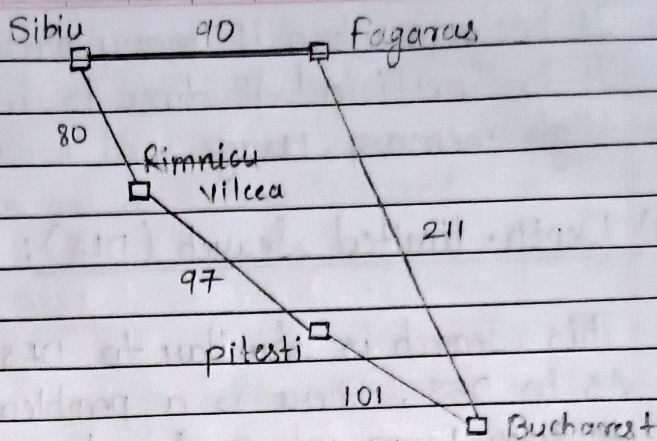
(C) Uniform-cost Search (UCS):

- It is a search algorithm used to find least-cost path, from the start node to the goal node.
- It is similar like BFS but it considers path cost instead of depth.
- In UCS path cost is represented as $g(n) \rightarrow$ total cost from a root node to node n .
- It tries to find better paths as compared to BFS.
- It uses a priority queue based on path cost.

• Properties of UCS:

- Completeness: Yes (if b is finite).
- Time and Space complexity: $O(b^{C^*})$ where C^* is the cost of optimal solution and b is the branching factor.
- Optimal: Yes, as it tries to find least-path cost.

• Example:



- The problem is to get from Sibiu to Bucharest. The successors of Sibiu are Rimnicu Vilcea and Fagaras, with costs 80 and 99; respectively. The least-cost node, Rimnicu Vilcea, is expanded next, adding Pitesti with cost $80 + 97 = 177$. The least-cost node is now Fagaras, so it is expanded, adding Bucharest with cost $99 + 211 = 310$. Now a goal node has been generated, but Uniform-cost Search keeps going, choosing Pitesti for expansion and adding a second path to Bucharest with cost $80 + 97 + 101 = 278$. Now the algorithm checks to see if this new path is better than the old one; it is, so the old one is discarded. Bucharest, now with g-cost 278, is selected for expansion and the solution is returned.

• Advantages:

- Find Optimal least-cost path.
- It works well when cost are different.
- complete $\rightarrow$ It is complete, if b is finite.

• Disadvantages:

- It becomes slow if many path have similar cost.
- It is inefficient if tree is infinite.
- High memory usage.

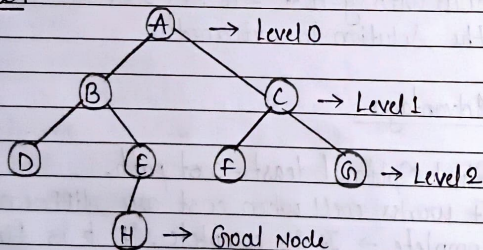
(d) Depth-limited Search (DLS):

- This Search is similar to DFS with little difference.
- As in DFS, there is a problem of reaching to infinite loop from where search cannot come back it stuck over them hence, no goal is found.
- To overcome the issues of DFS, DLS is invented.
- DLS also known as Depth-level Search.

• Properties of DLS:

- Completeness $\rightarrow$ doesn't guarantee to reach goal node.
- Time complexity: $O(b^L)$ (L depth limit).
- Space complexity: $O(bL)$
- Optimality: It does not give an optimal solution as it expands the nodes till the depth-limit.

• Example:



from the figure, H is a goal node. (Given) and A is a root node (given) and limit for search is given till Level 2. But our goal node is H and that is

Presented at level at level 3. According to limit it will search till level 2 only, hence goal-node is not found. This problem arises in DLS to solve this problem another method known as IDFS is used.

• Advantages of DLS:

- Depth limited Search is better than DFS and required less time and memory space.
- DFS assumes that the solution will be found if it exists infinite time.

• Disadvantages of DLS:

- This Search Strategy is not complete.
- It does not provide an optimal solution.

(e) Iterative Depth first Search (IDFS):

- IDFS is also known as IDDFS (Iterative deepening depth first Search).
- It is a Search Strategy that combines the benefits of DFS and BFS (completeness and Optimality for BFS and low memory use).
- This Search Strategy is used to solve the problem of DLS, as in DLS up till the limit given search is performance that's why, it is not necessary to reach a goal node.
- In IDDFS, if search limit is given till level 2 and if a goal node is at level 3 then we can increase the limit until solution is found.

Disadvantages:

- To Search from backward that is goal node, One should know the goal node in advance.
- It is not always possible to search from backward.

g) Comparing Uninformed Search Strategies:

Criteria	BFS	Uniform-cost	DFS	Depth-limited	IDS	Bidirectional
Complete?	Yes	Yes	No	No	Yes	Yes
Time	$O(b^d)$	$O(b^{C*/E})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{C*/E})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Optimal?	Yes	Yes	No	No	Yes	Yes

Where Notation are:

b - branching factor.

d - depth of the goal node.

m - maximum depth of search tree.

l - depth-limit.

e - lower bound of step cost.

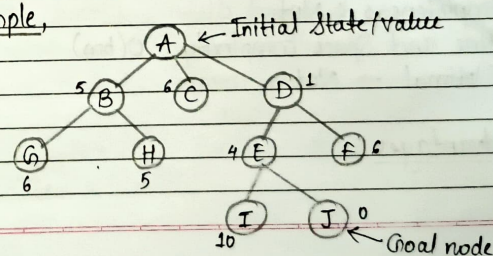
(5) Informed Search:

- An informed Search is more efficient than an Uninformed Search because in informed search along with the current state information, some additional information is also present, which make it easy to reach the goal state.
- Different types of informed search are:

1. Best first search (greedy search algorithm)
2. A* Search Algorithm
3. Memory bounded function
4. Learning to search better.

1. Best first search:

- It is also known as greedy search algorithm and it is a general approach for Informed search.
- In this search a node is selected for expansion based on an evaluation function $f(n)$, where $f(n)$ interprets the cost estimate value.
- The evaluation function expands that node first which are lower cost.
- A component of $f(n)$ is $h(n)$ which carries the additional information required for the search algorithms.
- $h(n)$ = estimated cost of the cheapest path from the current node n to the goal node.
- Note: If the current node n is goal node, the value of $h(n)$ will be 0.
- It is known as greedy search because it always tries to explore the node which is nearest to the goal node and selects that path, which gives a quick solution. Thus, it evaluates nodes with the help of the heuristic function that is, $f(n) = h(n)$.
- Example,



- Heuristic Value $h(n)$: It is estimated value or node value which is gained from experience (already given).

(a) Open list: It is Expanded node, the node which is not evaluated is known as Open list.
(Arrange increasing order of $h(n)$).

(b) Closed list: The Expanded node which is evaluated, traversed and becomes final Node is called closed list. (No need to Visited again).

Open list	Closed list
(D) (B) (C)	(A)
(E) (B) (C) (F)	(A) (D)
(J) (B) (C) (F) (I)	(A) (D) (E)

reach to
goal node

Hence,

OL $\rightarrow$ B C F I

CL $\rightarrow$ A D E J

• Properties:

- completeness: Not
- Time and Space complexity: $O(bm)$
- Optimal $\rightarrow$ Not

• Advantages:

- It takes more the advantage of DFS and BFS.
- This algorithm is more efficient than DFS and BFS.

• Disadvantages:

- It can get stuck in a loop as DFS.
- This algorithm is not complete or Optimal.

2. A* Search Algorithm:

- It is a type of informed search algorithm.
- It is very popular informed search algorithm used in path finding and graph traversal.
- Its find the shortest path from the start node (Initial node) to a goal node using heuristic function.
- It combines the features of UCS and greedy best-first search, by which it solve problem efficiently.
- It makes use of functions like $f(n)$, $h(n)$ and $g(n)$.
- It is Optimal algorithm and it follows backtracking rule which means it can re-visited previous state.

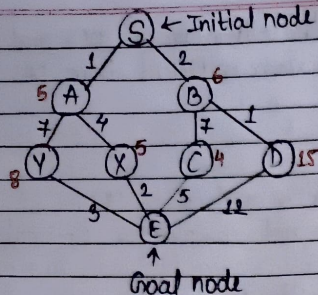
$$f(n) = g(n) + h(n)$$

Estimated cost
of the cheapest
solution.

functions that
tells how much
we have travel
from initial
node.

Heuristic value
or Estimated
value

- Examples,



Heuristic Value:
(Given)

$A \rightarrow 5$

$B \rightarrow 6$

$C \rightarrow 4$

$D \rightarrow 15$

$X \rightarrow 5$

$Y \rightarrow 8$

Step 1: Start from 'S' → Initial Root and expand it.

Step 2: 'S' is explore into two part A and B.

Step 3: $(S, A) :- f = 1 + 5 = 6$ and

$(S, B) :- f = 2 + 6 = 8$.

Step 4: It follows backtracking rules,

Step 5: After expanding 'S' we get that Value of node 'A' is less, so now we will expand 'A'; But will keep as $(S, B) :- f = 2 + 6 = 8$.

Step 6: Now expanding 'A' we get;

$(S, A, X) f = (1 + 4) + 5 = 10$ and

$(S, A, Y) f = (1 + 7) + 8 = 16$,

$(S, B) f = 2 + 6 = 8$.

Step 7: After comparing all the above 3 Values, we get that node 'B' is less (Using Backtracking rules). Now, Expanding 'B' we get node C and D.

Step 8: $(S, B, C) = f(n) = (2 + 7) + 4 = 13$.

$(S, B, D) = f(n) = (2 + 1) + 15 = 18$.

$(S, A, X) = f(n) = (1 + 4) + 5 = 10$.

$(S, A, Y) = f(n) = (1 + 7) + 8 = 16$.

After comparing above all 4 Values, we get that node 'X' is less/least. Now, expanding 'X' node

we get only one node that is 'E' → goal Node.
therefore, $(S, A, X, E) = f(n) = (1 + 4 + 2) + 0 = 7$.
∴ The best path is $(S \rightarrow A \rightarrow X \rightarrow E)$ with actual cost is 7.

• Properties of A* Algorithm:

(1) Completeness :- Yes

(2) Optimality :- Yes

(3) Time complexity :- $O(b^m)$

(4) Space complexity :- $O(b^m)$

• Advantages:

- It is considered best algorithm among other search technique, Use to solve complex problem.
- It is complete and Optimal.

• Disadvantages:

- This algorithm is complete if the branching factor is finite and every action has fixed cost.
- It has complexity problems.
- It is mainly depending on heuristic Value.

3. Memory bounded functions:

- It is type of Informed search technique / algorithm.
- This algorithm limit the memory usage while searching for solution in a problem space.
- It's try to find best part using limited memory.
- Example, Algorithms are used like :-

- (a) RBFS (Recursive best first Search)
 (b) IDA* (Iterative Deepening A* Search)
 (c) SMA* (Simplified Memory $\times$ A* Search)
 Bound

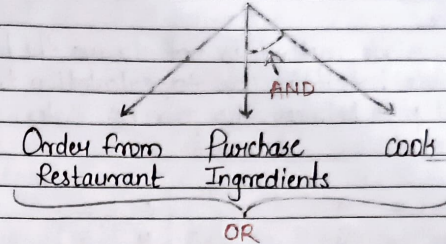
4. Learning to Search Better:

- In AI learning to search better means teaching the computer to improve how it's search for solutions overtime.
- Intelligent Agent search through many possible solutions like games or problem solving.
- If it learns from experience then it can avoid bad paths, choose better options first and reach the goal faster.

(6) AO* Search Algorithm:

- It is a type of informed search algorithm.
- This search is a specific specialized graph based on AND / OR Operations.
- It is a problem decomposition strategy where, a problem is decomposed into smaller pieces and solved separately to get a solution required to reach the desire goal.
- A* Search and AO* Search both follow best-first search order.
- Example,

Want to eat food



- Here, the destination/goal is to eat some food. We have two ways, either order food from any restaurant or buy some food ingredients and cook to eat food. Thus, we can apply any of the two ways, the choice depends on us. It is not guaranteed whether the order will be delivered on time, food will be tasty or not, etc. But if we will purchase and cook it, we will be more satisfied.
- Therefore, the AO* Search provides two ways to choose either OR or AND. It is better to choose AND rather OR to get a good optimal solution.

Heuristic function:

- It is a function which is used in Informed Search, and it finds the most promising path.
- In the function, it is denoted by symbol ' $h(n)$ ', also known as node value which is gain by and estimated by experience.
- It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal state.

- (4) The Value of the heuristic function is always positive.
- (5) This heuristic method is not always the best to find goal node but, it is use to calculation the cost of Optimal path between the pair of States.
- (6) Example,

7	2	4
5		6
8	3	1

Initial State

	1	2
3	4	5
6	7	8

Goal State

Step 1: Total moves (Nodes) from first node or function that is, $h_1 = 8$.

Step 2: To find h_2 we have to calculate particular node to bring it at proper position. How many moves it requires to reach goal state.

$h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$ (--- no. of moves required to reach the goal nodes).

Steps: for getting Optimized solution, we have to add both the function h_1 and h_2 , therefore $h_1 + h_2 = 8 + 18 = 26$.

True cost = 26 (Maximum moves required to reach the goal Node).

chap 4: Beyond Classical Search

(1) Local Search Algorithm and Optimization Problem:

- The Search algorithms till now that we have studies is use to explore and take actions in order to reach the goal state.
- A Local Search Algorithm is use to find a good sol by starting from one possible solution and then moving step-by-step to get better more optimal solutions.
- It doesn't explore all solutions, only the near by states.
- Different types of Local Search algorithms are :-
 - (a) Hill Climbing (Steepest ascent/descent)
 - (b) Simulated Annealing: Inspired by Statistical physics.
 - (c) Local beam search
 - (d) Genetic algorithm: Inspired by evolutionary biology.
- Optimization problem is When we need to find best solution from many possible solutions based on conditions like least path cost, less searching time, less cost and even reaching to goal state.

(A) Hill Climbing Algorithm:

- It is a local search algorithm that try's to find the best (highest or lowest) solution by making small changes to the current solution.
- It keep's moving in the direction of improvement until no better solution's is find / found.

• Algorithm:

- Step 1: Start With the initial solution.
- Step 2: Evaluate its quality (fitness).
- Step 3: Look at the near by solutions.
- Step 4: Move to the better neighbour.
- Step 5: Repeat Until no better neighbour exist.

• Features of Hill climbing:

- (a) Generate and test Variant.
- (b) Greedy Approach.
- (c) No backtracking.

IMP ASS • State-Space Diagram for Hill climbing:

• Types of Hill Climbing Algorithm:-

- (a) Simple hill climbing: It is the simplest way to implement hill climbing algorithm.
 - It only evaluates the neighbor node state at a time and selects the first one which optimizes current cost and set it as a current state.
- (b) Steepest-Ascent hill climbing: It is a variation of simple hill climbing algorithm.
 - This algorithm examines all the neighbouring nodes of the current state and selects one neighbor node which is closest to the goal state.
- (c) Stochastic hill climbing:

- It doesn't examine for all its neighbor before moving. Rather, this search algorithm selects one neighbor node at random and decides whether to choose as a current state or examine another state.

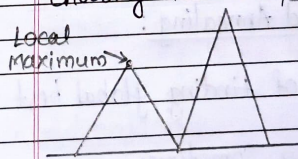
IMP • Problems in Hill climbing Algorithm:

- (a) Local Maximum: It is a peak state in the landscape which is better than each of its neighboring states, but there is another state also present which is better than the local maximum. That's why this algorithm gets stuck at this peak which is not highest.

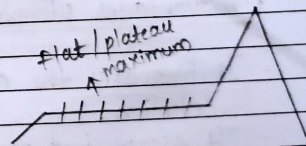
- Solution: Backtracking technique is a solution to find another best highest peak.

- (b) Plateau: A plateau is the flat area of the search space in which all the neighbor states of the current state contains the same value, because of this algorithm doesn't find any best direction to move. A hill climbing search might be lost in the plateau area.

- Solution: Stochastic hill climbing search is used in which we take a random move instead of always choosing the best part.



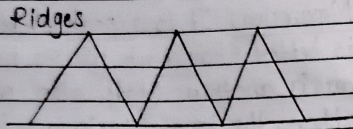
(a) Local Maximum



(b) Plateau

(C) Ridges: A ridge is a special form of the local Maximum. It has an area which is higher than its surrounding areas, but itself has a slope, and cannot be reached in a single move.

- Solution: With the use of bidirectional search, or by moving in different directions, we can improve this problem.



(B) Simulated Annealing:

- It is a type of local search algorithm, used to solve optimization problems.
- In mechanical terms, annealing is a process of hardening a metal or glass to a high temperature then cooling gradually, so this allows the metal to reach a low-energy crystalline state. The same process is used in simulated annealing, in which the algorithm picks a random move, instead of picking the best move, if the random move improves the state, then it follows the same path.

• Advantages of Simulated Annealing:

- It increases the chance of finding global best solution.
- It escapes bad moves, for sometimes.

• Algorithm:

Step 1: Start with an initial solution (current).

Step 2: Set an initial temperature (T).

Step 3: Repeat until T is very low:

- Make a small random change to get new-solⁿ.
- Calculate the change in cost:
 $\Delta = \text{cost}(\text{new-sol}^n) - \text{cost}(\text{current})$.
- If $\Delta < 0$, accept new-solⁿ (it's better).
- Else, accept with probability $e^{(-\Delta/T)}$.
- Reduce temperature ($T = T \times \text{cooling-rate}$).

Step 4: Return the best solution found.

(C) Local Beam Search:

- It is a local search algorithm that keeps track of multiple solution (states), at the same time instead of just one.
- It tries to find the best solution by always keeping the top 'K' best states, during the search.

• Algorithm:

Step 1: Choose 'K' random initial state.

Step 2: Repeat:

- each state, generate all possible neighbors.
- keep the 'K' best neighbor out of all.

Step 3: If any state is a goal, return it.

Step 4: Stop if no better states are found.

• Advantages:

- It is more efficient than random search.

- It Uses multiple paths and find best One Out Off it.

• Disadvantages:

- If 'k' States become Same, then it will stuck like hill climbing.

(D) Genetic Algorithm:

- It is best and Optimization technique based on the process of natural selection. (Darwin's theory: Survival of the fittest).

- It is used to find good solution to the complex problems where normal search methods are slow or get stuck.

- Steps of GA (genetic algorithm):

1. Initialize Population:

- Start with a set of random solutions (called chromosomes).

2. Evaluate fitness:

- check how good each solution is (fitness function).

3. Selection:

- choose the best solutions (parents) based on fitness.

4. Crossover (Reproduction):

- combine two parents to make new children (new solutions).

5. Mutation:

- Randomly change parts of a child to maintain diversity.

6. Replace:

- form a new generation using the new children.

7. Repeat:

- Continue steps 2-6 until the best solution is found.

(2) Searching With Non-deterministic Action:

- In this search, an Environment is Unknown for an Agent, so it is not sure that whatever action is going to take, it is not whether it search or not, that's why only contingency plan is needed.

(A) The Erratic Vacuum World:-

- It is a variation of basic Vacuum cleaner world, and use of represent concepts in intelligent agent, reflects agent and search algorithm.

- It consists of either 2 states or 8 states and 4 actions:-

• left • right • Suck • Nope

- Erratic means, Agent Action don't always go as intended.

(B) AND-OR Search trees:-

- This search tree is use to solve a problem.

- This search includes BFS, DFS and best first search. This is a new type of search tree, it is use to provided a solution to non-deterministic problem, by using two nodes: AND or OR.

(C) Try, try again (Slippery) (non-erratic) Vacuum world:-

- Date _____
Page _____
- It is another Variation of Vacuum cleaner - world agents. It is Use to teach behavior of agents in deterministic but partially observable environment, with action failure, yet Without randomness in the action type.
 - As it is Slippery actions may fail, means agent may not moves, even it try's. So agent needs to try again if it action fail.

(3) Searching With Partial Observations:

- An Agent is going to search for an goal state depending upon its partial Observations.
- It includes 4 Scenarios:

(A) Searching With no-Observation:

- An agent is Sensorless. example, Vacuum cleaner world With no Sensors.
- Then it works on its belief State.

(B) Searching With Observation:

- An agent consists of Sensors, going to take an input from the environment, and shows action again in an environment.

(C) Searching With Partial Observable Problem:

- An agent knows partially about the environment, based on perform search or action to reach goal state. Example, AND-OR Search Algorithm is used.

(D) An agent for Partially Observable environment:

- An agent in partially Observable environment must act based upon incomplete or uncertain information. Since, it can't see the full state of the environment, its need memory, reasoning, or Sensing strategies to make intelligent decisions.
- Example, conditional Statements like, if, then, else Statement is used.

(4) Online Search Agents and Unknown Environments:

- An Online Search agent interleaves computation and action. first it takes an action, then it Observes the environment and computes the next action.
- Online Search is a good idea in dynamic or semi-dynamic domains/agents.

- It includes:-

(a) Online Search Problems: Agent doesn't known the map learns while moving.

(b) Online Search Agents: These agents make decision step-by-step with limited knowledge, example, A*, UCS.

(c) Online Local Search: It uses nearby information to choose the next move it takes random walk.

(d) Learning in Online Search: In such case an agent has to remember its past actions to do better in future moves. That's Why an agent keeps on learning to update its prior knowledges.